

OPERATOR'S MANUAL

SHARPTUI

Retained tree terminal interface framework

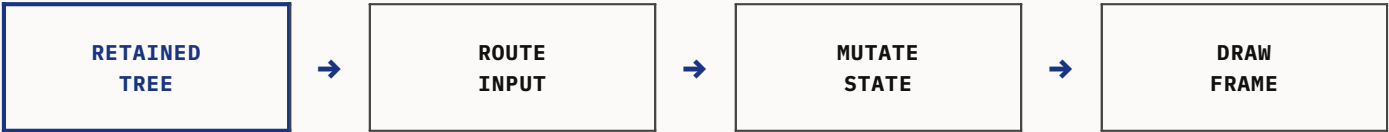


FIGURE 0-1. NORMAL APPLICATION OPERATING CYCLE

SUBJECT	APPLIES TO	LANGUAGE
INITIAL OPERATION	SHARPTUI APPLICATIONS	G# / .NET 10

DISTRIBUTION	FOR APPLICATION AUTHORS, MAINTAINERS, AND TEST OPERATORS. KEEP WITH THE CURRENT API REFERENCE.
--------------	--

00

FRONT MATTER

MANUAL CONTROL

SCOPE

This manual gives the minimum operating sequence for a SharpTUI application. It covers tree construction, view routing, state mutation, frame drawing, and deterministic verification.

REVISION RECORD

REV.	DATE	DESCRIPTION
A	2026-08	Initial issue

READING NOTATION

CAUTION	Prevents invalid application behavior.
NOTE	Adds an operating detail or boundary.
LIST.	Provides source text for direct use.
FIG.	Shows routing or state sequence.

TABLE OF CONTENTS

00	Manual control	ii
01	Run a retained tree	1-1
02	Operate through a view	2-1
03	Test the operating path	3-1
A	Operating prohibitions	A-1

ILLUSTRATION AND LISTING REGISTER

0-1	Normal application operating cycle	Cover
1-1	Minimal retained tree	1-1
2-1	State-owning view	2-1
2-1	Application input route	2-2
3-1	Deterministic driver sequence	3-1

REFERENCE	For every public member, use the current API reference.
-----------	---

01

CHAPTER 1

RUN A RETAINED TREE

1.1 Operating principle. SharpTUI retains a tree. Construct it once. Mutate the same objects when state changes. Draw the tree into a `Screen` after each event batch.

CAUTION

Do not rebuild the tree in `Draw`. Do not draw application state only into a status line. Change the widget that owns the state.

1.2 `App.Run(root Box)` owns the terminal session. It runs until a handler returns `EventResult.Exit` or a fallback quit gesture matches. New apps use Escape and Ctrl+C as fallback quit gestures.

TYPE	ROLE	OPERATING FACT
<code>App</code>	Terminal loop	Runs a <code>Box</code> tree or a custom <code>View</code> .
<code>Box</code>	Retained node	Every drawable control and container derives from it.
<code>Row</code>	Horizontal container	Lays children on the row axis.
<code>Column</code>	Vertical container	Lays children on the column axis.
<code>Screen</code>	Cell frame	Receives clear, write, fill, and border calls.

LISTING 1-1

MINIMAL RETAINED TREE

```
package Demo

import SharpTui

func Main(args []string) int32 {
    let root = Column{
        Padding: CellInsets.All(1),
        GapCells: 1,
        Children: {
            Label{ Text: "SharpTUI" },
            Row{
                GrowWeight: 1,
                Children: {
                    Box{ Width: CellLength.Cells(20), ShowBorder: true, Title: "left" },
                    Box{ GrowWeight: 1, ShowBorder: true, Title: "right" },
                },
            },
        },
    },
    app := App()
    app.Run(root)
    return 0
}
```

1.3 `CellLength.Cells(20)` uses a fixed 20-cell width. Its count must be 0 or more. `GrowWeight` distributes spare space only within its immediate in-flow parent.

02

CHAPTER 2
OPERATE THROUGH A VIEW

2.1 **View** is for an application that owns state and a retained tree. **Draw(screen)** paints one frame. **Handle(ev)** returns **Continue**, **Handled**, or **Exit**.

RESULT	MEANING	NEXT STAGE
Continue	The view declined the event.	Later routing may run.
Handled	The view consumed the event.	Routing stops.
Exit	The app must stop.	The loop exits.

LISTING 2-1

STATE-OWNING VIEW

```
package Demo

import SharpTui

class Counter : View {
    private var root Box
    private var value Label
    private var count int32

    public init() {
        count = 0
        value = Label{ Text: "0" }
        root = Column{
            Padding: CellInsets.All(1),
            Children: { Label{ Text: "press +" }, value },
        }
    }

    public func Draw(screen Screen) {
        screen.Clear()
        root.Draw(screen)
    }

    public func Handle(ev UiEvent) EventResult {
        let routed = root.Handle(ev)
        if routed != EventResult.Continue { return routed }
        if KeyGesture.Character("+").Matches(ev) {
            count = count + 1
            value.Text = count.ToString()
            return EventResult.Handled
        }
        return EventResult.Continue
    }
}

func Main(args []string) int32 {
    let app = App()
    app.Run(Counter())
    return 0
}
```

02

CHAPTER 2 CONTINUED

ROUTE AND SCHEDULE WORK

2.2 The app offers `BeforeWidgets` first. The view receives an event only when that phase declines it. Inside a custom view, route the retained root before view fallback shortcuts. Mutating `value.Text` retains the new state.

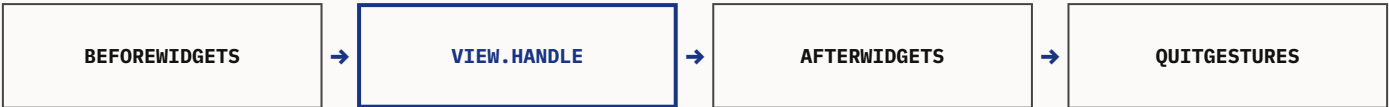


FIGURE 2-1. APPLICATION INPUT ROUTE

2.3 `App.StartWorker[T](work)` owns one background operation and returns its terminal state to the application loop. Use `App.Post(work)` for results from external producers. `App.RequestDraw()` requests one coalesced repaint.

Routing and update register

OPERATION	OWNER	REQUIRED ORDER
<code>BeforeWidgets</code>	Application keymap	Run before the view.
<code>View.Handle(ev)</code>	Application view	Route the root before view fallback shortcuts.
<code>AfterWidgets</code>	Application keymap	Run after the view returns <code>Continue</code> .
<code>Quit Gestures</code>	Application loop	Run last.
<code>StartWorker[T]</code>	Application loop	Own background work and terminal delivery.
<code>App.Post(work)</code>	External producer	Deliver results to the loop.
<code>RequestDraw()</code>	Application loop	Request a draw when input did not.

Normal operating sequence

- 01 Offer `BeforeWidgets` application commands.
- 02 Call `View.Handle`. A custom view routes its retained root before its fallback logic.
- 03 Offer `AfterWidgets` only after the view returns `Continue`.
- 04 Offer `Quit Gestures` last.
- 05 Draw the next dirty frame after accepted state changes.

CAUTION

Use `App.StartWorker` for app-owned work. Return external results through `App.Post`. Never mutate the retained tree from a worker thread.

03

CHAPTER 3

TEST THE OPERATING PATH

3.1 `TestDriver` runs a `Box` tree or `View` without a terminal or clock. Its width and height are terminal cells. Both values must be 0 or more.

METHOD	RESULT
<code>Send(ev)</code>	Routes one event through the normal application pipeline.
<code>Pump()</code>	Runs posted work and samples animations at the current time.
<code>Draw()</code>	Paints and returns the ANSI diff.
<code>Resize(width, height)</code>	Changes the deterministic screen size.
<code>Advance(elapsed)</code>	Moves caller-controlled time forward.

LISTING 3-1

DETERMINISTIC DRIVER SEQUENCE

```
let driver = TestDriver(Counter(), 40, 8)
let result = driver.Send(UiEvent{
  Kind: UiEventKind.Key,
  Key: Key.Character,
  Text: "+",
  Phase: KeyPhase.Press,
})
let frame = driver.Draw()
```

3.2 The driver exposes its `App`, `Screen`, current `NowMilliseconds`, and the focused element for a driven tree.

Standard verification sequence

- 01 Construct the retained tree or state-owning view.
- 02 Create the driver with a fixed cell width and height.
- 03 Send the same event used during terminal operation.
- 04 Draw the frame and inspect the resulting screen state.
- 05 Advance caller-controlled time only when the test requires it.

NOTE

A driver test uses the application routing path. It is not a separate widget simulation.

APPENDIX A

A

OPERATING PROHIBITIONS

The following conditions cause incorrect ownership, routing, or rendering behavior. Observe each prohibition during application construction and maintenance.

A-01	Don't recreate widgets to show a value change. Mutate the retained widget.
A-02	Don't mutate the tree from a background thread. Use <code>App.StartWorker</code> for app-owned work or <code>App.Post</code> for external results.
A-03	Don't omit <code>screen.Clear()</code> before drawing a custom <code>View</code> unless the view deliberately owns every cell.
A-04	Don't return <code>Handled</code> for an event the view did not consume.

RELATED PUBLICATIONS

STUI-REF	Generated API reference. Member signatures and anchors.
STUI-TXT	Text and rich content. Text presentation and document controls.
STUI-COL	Collections. Bound collection operation.
STUI-CTL	Controls and overlays. Focus, selection, and overlay state.
STUI-CAN	Canvas, animation, and work. Timed drawing and background results.

SOURCE ISSUE

The editable source for this publication is `getting-started.md`. When code or behavior changes, update the source guide, this manual issue, and the API reference in the same review.

END OF MANUAL	SHARPTUI TECHNICAL PUBLICATIONS / STUI-01 / REVISION A
---------------	--